

itertuples()

Technical Design and Performance Analysis

Overview

Satellite position calculation requires iterating through every observation record in the dataset. Because this operation occurs for every row, even small inefficiencies in row traversal can significantly impact overall runtime.

This optimization focused on reducing per-row processing overhead by replacing Pandas' `apply(axis=1)` implementation with an `itertuples()`-based approach in both the multiprocessing worker path and the single-core fallback path.

The goal was to improve execution efficiency without modifying the underlying satellite position calculation logic, ensuring that performance gains could be achieved while preserving correctness and maintainability.

Original Implementation

The original implementation relied on Pandas' row-wise apply functionality.

```
self.obs.apply(  
    lambda x: self.get_satellite_position(x),  
    axis=1,  
    result_type='expand'  
)
```

This approach is straightforward and expressive, but it introduces additional work for every row processed.

Each observation must first be converted into a Pandas `Series` object before being passed to the calculation function.

Although convenient from a coding perspective, this abstraction introduces overhead that becomes increasingly significant as dataset size grows.

Identified Performance Problems

1. Series Construction Overhead

When using:

```
apply(axis=1)
```

Pandas constructs a full `Series` object for each row before invoking the callback function.

For every observation, Pandas performs:

1. Row extraction
2. Series construction
3. Function invocation
4. Result collection

The actual satellite-position calculation is only one portion of the work being performed.

As observation counts increase, the cumulative cost of repeatedly constructing temporary `Series` objects becomes substantial.

2. Repeated Object Allocation

Large observation datasets may contain thousands or millions of rows.

Constructing a `Series` object for every row introduces:

- Additional memory allocations
- Additional garbage collection pressure
- Additional interpreter overhead

These costs accumulate throughout execution and can consume a meaningful portion of total runtime, particularly when the calculation itself is already optimized.

3. Fallback Performance Limitations

The same row-processing approach was used in the single-core fallback path.

As a result, systems unable to use multiprocessing experienced additional overhead beyond the computational work required for satellite calculations.

This meant that the fallback implementation inherited the same inefficiencies as the parallel implementation.

Optimization Strategy

Optimization: Replace `apply(axis=1)` **with** `itertuples()`

The implementation was refactored to iterate directly over rows using:

```
[
    self.get_satellite_position(row)
    for row in self.obs.itertuples()
]
```

and similarly within worker processes.

Unlike `apply(axis=1)`, `itertuples()` produces lightweight named tuples rather than constructing full `Series` objects.

Conceptually:

```
Row -> NamedTuple -> Calculation
```

instead of:

```
Row -> Series Construction -> Calculation
```

This reduces the amount of work required before the position calculation can begin.

Benefits

Reduced Object Construction

Named tuples require substantially less overhead than full `Series` objects.

This reduces:

- Memory allocation costs
- Object creation costs
- Per-row processing overhead

while preserving access to row data in a structured format.

Faster Iteration

`itertuples()` is optimized specifically for row traversal and avoids much of the machinery associated with Pandas row-wise apply operations.

Because fewer intermediate objects are created, iteration becomes more efficient and places less burden on the Python interpreter.

Consistent Improvement Across Execution Modes

The optimization was applied to:

- Multiprocessing workers
- Single-core fallback execution

This ensures performance improvements are realized regardless of whether parallel execution is available.

The behavior of the calculation remains identical, while execution efficiency improves in both scenarios.

Design Considerations

The refactor intentionally focused only on the row-iteration mechanism and did not alter the satellite-position calculation logic itself.

By isolating the change to the data-access layer, the optimization reduces risk while providing measurable performance benefits.

Comments were included to document why `itertuples()` was selected and how its behavior differs from `apply(axis=1)`, making the reasoning behind the change transparent to future maintainers.

This approach preserves readability while improving performance, allowing future contributors to understand both the implementation and the rationale behind the optimization.